

Name: ycchiu226

Date: 2026/09/25

Topic: Recursion — Nested Structures and Problem Reduction

1. Today's Challenge

Core concept from today's OCW lecture:

Recursion — reducing a problem to a smaller version of itself, using a base case and recursive case. I also applied the idea of nested recursive structures.

AI-generated coding challenge title:

Nested Message Decoder

2. My Initial Approach — Before AI Help

Before asking AI for hints, briefly describe how you planned to solve the problem.

My approach:

I planned to recursively process the string from the first opening parenthesis. I would strip the string after the first `(`, recursively call `parentheses_decoder()` on the remaining substring, and then process the corresponding `)` after the recursive call returned. I would add 1 to the returned depth for the current level and keep the depths in a list, finally returning the maximum depth.

3. AI Tutor Help

Did you ask the AI Tutor for help?

- ☒ No — I solved it independently
☐ Yes — I received one or more hints

The most useful hint/question from AI was:

It helped me realize that:

4. My Revision

Did you change your approach or code after interacting with AI?

☒ No

☐ Yes

What did you change, and why?

5. Verification

My final program:

☒ Passed the provided examples

☒ Passed additional edge cases

☐ Still has unresolved problems

One edge case I tested:

Input: `a()((b(c)d)e)`

Expected output: `3`

Actual output: `3`

I also tested cases such as `abc`, `a(b)c`, `(( ))`, `(( ( )))`, and `(( ))`.

6. One-Minute Reflection

What idea from the OCW lecture did I transfer to this new problem?

I transferred the idea of a nested recursive structure and reducing a problem to a smaller version of itself. Instead of reproducing the lecture examples, I applied the same recursion pattern to finding the maximum nesting depth of parentheses.

One thing I understand better now:

I understand more clearly that a recursive call should solve a well-defined smaller version of the same problem, and that the result can be combined with the current level after the recursive call returns. I also understand that the implementation cost of string operations matters when analyzing time complexity.

One thing I am still unsure about:

I am still developing my understanding of time complexity when Python string operations such as `find()` and `split()` are used. In particular, I understand that they stop after the first match when appropriate, but I want to become more confident in determining their exact cost and how repeated substring processing affects the overall complexity.